

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: V S SRAVAN M

Roll No.: A24126552268

1. TITLE

Build an Interactive Flashcard App using HTML, CSS and JavaScript

2. AIM

To build an interactive, single-page flashcard web application using HTML for structure, CSS for modern card styling and transitions, and JavaScript for DOM manipulation, state management, and event handling.

3. OBJECTIVE

The objectives of this experiment are:

- To integrate HTML structure, CSS styling, and JavaScript logic into a unified web application.
 - To design an interactive card UI featuring drop shadows, rounded borders, and hover elevation.
 - To query and select DOM elements using `document.getElementById()`.
 - To bind user click interactions using event listeners (`addEventListener`).
 - To maintain and toggle application state (`currentIndex`, `isShowingAnswer`).
 - To dynamically update DOM text and toggle CSS classes (`textContent`, `classList.add`, `classList.remove`).
 - To implement cyclic array navigation using modulo arithmetic $((\text{currentIndex} + 1) \% \text{flashcards.length})$.
-

4. THEORY

Modern web development is built upon the interaction between the DOM (Document Object Model), CSSOM (CSS Object Model), and JavaScript execution. Event-driven architecture enables applications to respond dynamically to user inputs such as mouse clicks, keyboard presses, or touches.

Theory of Flashcard Application Architecture:

- **Document Object Model (DOM):** An object-oriented tree representation of web elements accessible and mutable via JavaScript.
- **Event Listeners:** `addEventListener()` binds event handlers to DOM nodes without overwriting other attached handlers.
- **Component State Management:** Managing in-memory variables (`currentIndex`, `isShowingAnswer`) that govern visual rendering.
- **Dynamic CSS Class Toggling:** `classList.add()` and `classList.remove()` trigger CSS animations and background color changes.

- **Cyclic Array Traversal:** Utilizing the modulo operator (%) to loop back to the initial card upon completing the question list.
-

5. ALGORITHM / PROCEDURE

1. Create project directories: Record/Experiment - 4(week -4)/Src and Documentation.
 2. Create index.html and script.js in the Src directory.
 3. In index.html, construct a centered layout with a flashcard element containing label, text, and hint, followed by a 'Next Question' button.
 4. Write embedded CSS for card styling, hover transformations, transition effects, and a green background state for revealed answers.
 5. Link the external script file via `<script src="script.js"></script>`.
 6. In script.js, define an array of flashcard objects each holding question and answer pairs.
 7. Initialize state variables: `currentIndex = 0` and `isShowingAnswer = false`.
 8. Select DOM elements by ID (`#flashcard`, `#cardLabel`, `#cardText`, `#nextBtn`).
 9. Define `loadCard()` function to render the current question, reset the label, and remove the revealed class.
 10. Add a click event listener on `#flashcard` to toggle between question and answer states upon clicking.
 11. Add a click event listener on `#nextBtn` to advance the card index cyclically and invoke `loadCard()`.
 12. Open the page in Google Chrome and verify card flipping and question progression.
-

6. PROGRAM

index.html - Part 1 (Lines 1-52)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Beginner Flashcard App</title>

  <style>
    /* 1. Overall Page Layout */
    body {
      font-family: Arial, sans-serif;
      background-color: #eef2f3;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
      margin: 0;
    }

    .container {
      text-align: center;
      width: 320px;
    }

    /* 2. Flashcard Styling */
    .card {
      background-color: white;
      border-radius: 12px;
      padding: 40px 20px;
      box-shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
      cursor: pointer;
      min-height: 120px;
      display: flex;
      flex-direction: column;
      justify-content: center;
      align-items: center;
      transition: transform 0.2s ease, background-color 0.3s ease;
    }

    .card:hover {
      transform: translateY(-3px);
    }

    .card-title {
      font-size: 0.85rem;
      color: #888;
      text-transform: uppercase;
      letter-spacing: 1px;
      margin-bottom: 10px;
    }

    .card-text {
```

index.html - Part 2 (Lines 53-106)

```
        font-size: 1.25rem;
        color: #333;
        font-weight: bold;
        margin: 0;
    }

    /* Active state when answer is showing */
    .card.revealed {
        background-color: #e8f5e9;
    }

    .hint-text {
        font-size: 0.8rem;
        color: #aaa;
        margin-top: 15px;
    }

    /* 3. Button Styling */
    .btn {
        margin-top: 20px;
        padding: 12px 24px;
        font-size: 1rem;
        background-color: #3498db;
        color: white;
        border: none;
        border-radius: 6px;
        cursor: pointer;
        width: 100%;
    }

    .btn:hover {
        background-color: #2980b9;
    }
</style>
</head>
<body>

<div class="container">
    <!-- The Flashcard -->
    <div class="card" id="flashcard">
        <span class="card-title" id="cardLabel">Question</span>
        <p class="card-text" id="cardText">Loading question...</p>
        <span class="hint-text">(Click card to flip)</span>
    </div>

    <!-- Next Question Button -->
    <button class="btn" id="nextBtn">Next Question</button>
</div>

<!-- Linking the JavaScript File -->
<script src="script.js"></script>

</body>
</html>
```

script.js (Lines 1-52)

```
// Step 1: Store our data (Array of question/answer objects)
const flashcards = [
  { question: "What does HTML stand for?", answer: "HyperText Markup Language" },
  { question: "What language styles a webpage?", answer: "CSS (Cascading Style Sheets)" },
  { question: "Which HTML tag connects a JS file?", answer: "<script src='...'>" },
  { question: "What tool does JavaScript use to listen for clicks?", answer: "addEventListener" }
];

// Track current question index & state
let currentIndex = 0;
let isShowingAnswer = false;

// Step 2: Select elements from HTML using IDs
const flashcardElement = document.getElementById("flashcard");
const cardLabel = document.getElementById("cardLabel");
const cardText = document.getElementById("cardText");
const nextBtn = document.getElementById("nextBtn");

// Step 3: Function to display current flashcard
function loadCard() {
  isShowingAnswer = false;
  cardLabel.textContent = "Question";
  cardText.textContent = flashcards[currentIndex].question;
  flashcardElement.classList.remove("revealed");
}

// Step 4: Event Listener - Flip Card on Click
flashcardElement.addEventListener("click", function () {
  if (isShowingAnswer) {
    // Flip back to Question
    cardLabel.textContent = "Question";
    cardText.textContent = flashcards[currentIndex].question;
    flashcardElement.classList.remove("revealed");
    isShowingAnswer = false;
  } else {
    // Show Answer
    cardLabel.textContent = "Answer";
    cardText.textContent = flashcards[currentIndex].answer;
    flashcardElement.classList.add("revealed");
    isShowingAnswer = true;
  }
});

// Step 5: Event Listener - Next Button Click
nextBtn.addEventListener("click", function () {
  // Move to next card index (loop back to start if at the end)
  currentIndex = (currentIndex + 1) % flashcards.length;
  loadCard();
});

// Load the first card when the page opens
loadCard();
```

7. CODE EXPLANATION

Method / Property	Explanation
<code>document.getElementById(id)</code>	Returns a reference to the DOM element with the specified ID attribute.
<code>addEventListener(event, fn)</code>	Attaches an event handler function to handle user click events.
<code>textContent</code>	Property used to get or update the text content of a DOM node.
<code>classList.add(cls)</code>	Adds a CSS class to the element's class list, triggering style changes.
<code>classList.remove(cls)</code>	Removes a CSS class from an element to revert to base styles.
Array of Objects	Stores flashcards as structured data ({ question: "...", answer: "..." }).
Modulo Operator (%)	Wraps the question index back to 0 when reaching the end of the array.
<code>transition: transform, bg</code>	CSS transition property enabling smooth state change animations.
<code>:hover transform</code>	CSS hover effect elevating the card slightly (-3px) on mouse over.
<code><script src="..."></code>	HTML tag importing and executing the external JavaScript file.

8. TEST CASES AND EXECUTION DATA

The following test suite was executed manually in Google Chrome to verify dynamic card interactions and event handling:

TC ID	Test Description	Input / Action	Expected Result	Status
TC-01	Initial Card Load	Open index.html in browser	Card renders Question 1: 'What does HTML stand for?' with Question label	Pass
TC-02	Card Click Flip to Answer	Click on flashcard body	Card label changes to 'Answer', displays answer text, and turns light green	Pass
TC-03	Card Click Flip to Question	Click on revealed flashcard	Card flips back to Question text and label, removing green background	Pass
TC-04	Next Question Navigation	Click 'Next Question' button	Advances to Question 2 ('What language styles a webpage?')	Pass
TC-05	State Reset on Navigation	Flip to Answer, then click 'Next Question'	New card appears in Question state with white background	Pass
TC-06	Circular Loop Navigation	Cycle through all 4 cards and click Next on last card	Card cyclically loops back to Question 1 seamlessly	Pass

9. OUTPUT

Output: Rendered Webpage in Chrome Browser

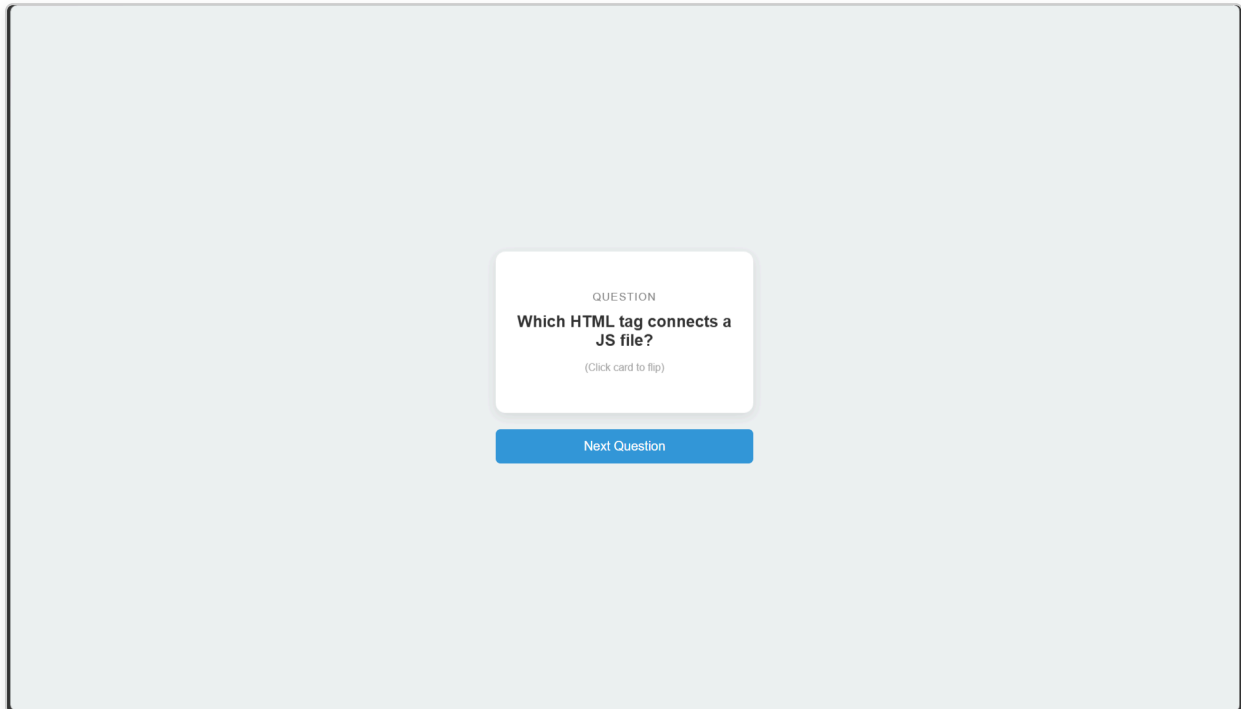


Figure 1: Initial state displaying Question 1.

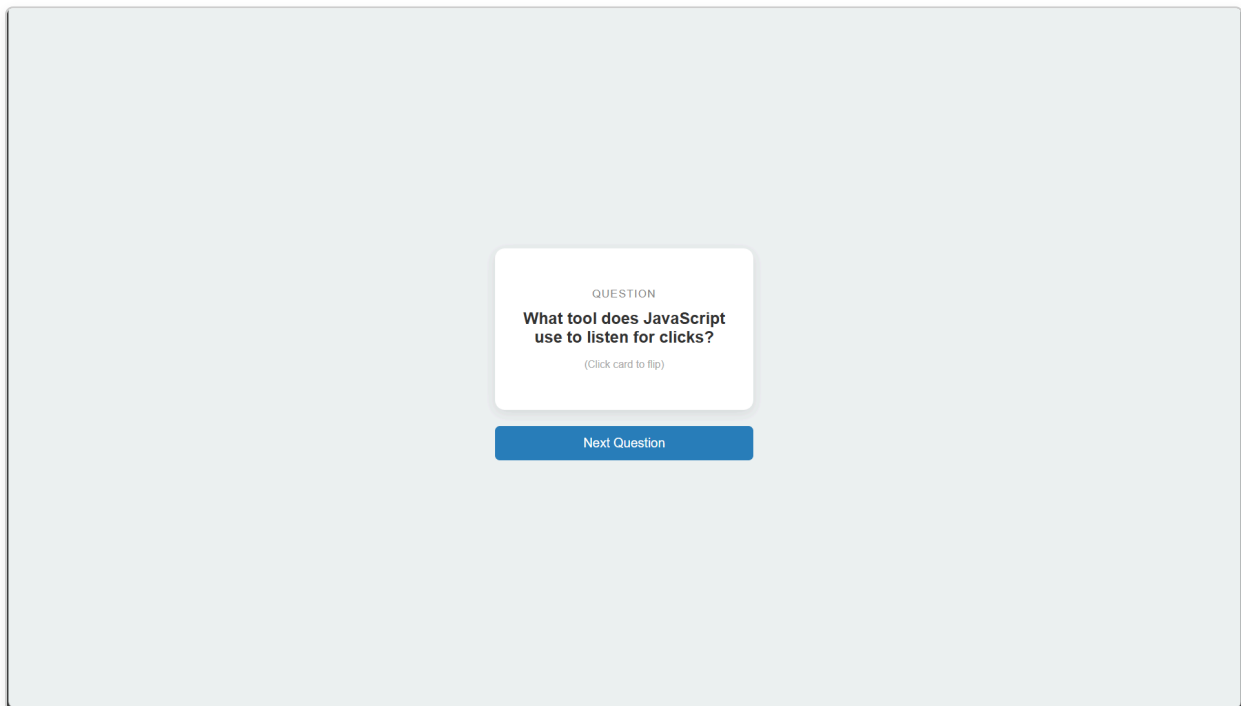


Figure 2: Flipped state displaying Answer with green background.

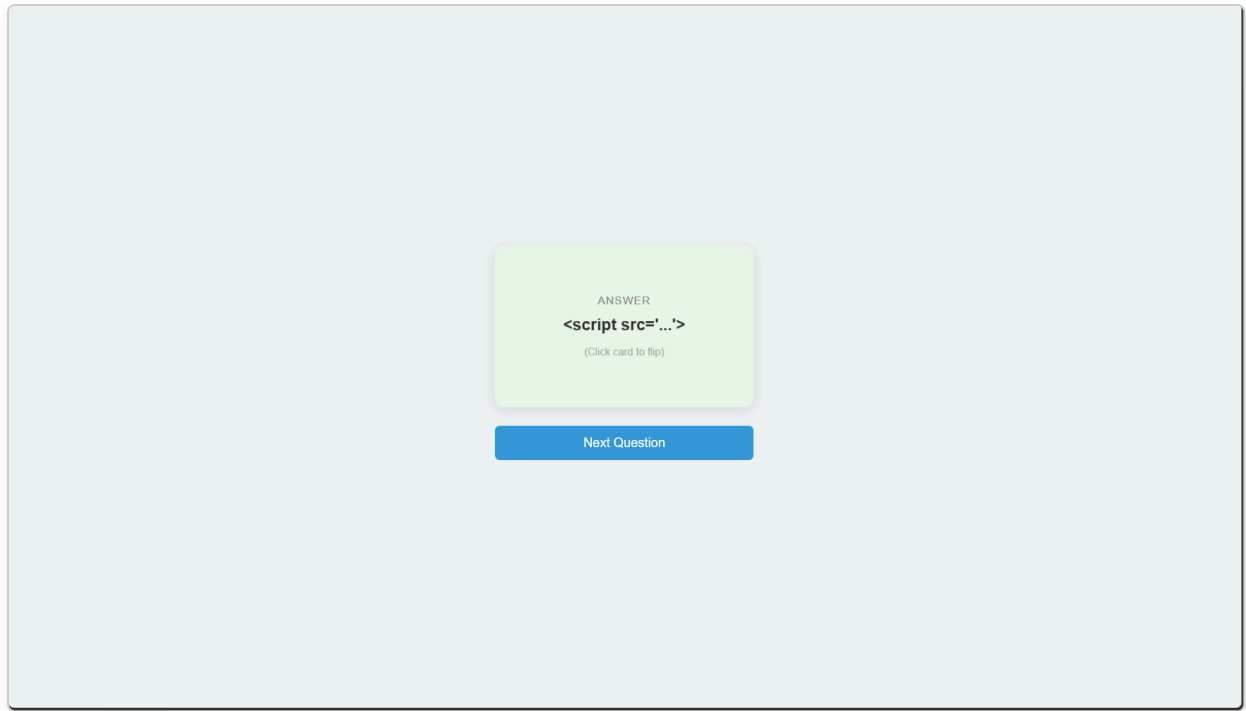


Figure 3: Next card state displaying Question 2.

10. RESULT

Thus, an interactive flashcard application utilizing HTML5, CSS3, and JavaScript DOM event handling was successfully designed, implemented, and verified using manual test cases under browser defaults.

Submission Package Structure:

```
Experiment - 4(week -4)/
├── Documentation/
│   ├── Experiment_4.docx
│   └── Experiment_4.pdf
├── Screenshot/
│   ├── Screenshot 2026-08-16 224206.png
│   ├── Screenshot 2026-08-16 224214.png
│   └── Screenshot 2026-08-16 224223.png
└── Src/
    ├── index.html
    └── script.js
```